

MSc Data Science Project 7PAM2002

Department of Physics, Astronomy and Mathematics

Data Science FINAL PROJECT REPORT

Project Title:

Safe Tweet Detector: Multi-Class Harm Detection using
Machine Learning and Deep Learning

Student Name and SRN:

Stephen Ogundero 24086166

Supervisor: Dhairya Kataria

Date Submitted:

Word Count: 4208

Github: [SafeTweetDetector](#)

DECLARATION STATEMENT

This report is submitted in partial fulfilment of the requirement for the degree of Master of Science in Data Science at the University of Hertfordshire.

I have read the detailed guidance to students on academic integrity, misconduct and plagiarism information at [Assessment Offences and Academic Misconduct](#) and understand the University process of dealing with suspected cases of academic misconduct and the possible penalties, which could include failing the project or course.

I certify that the work submitted is my own and that any material derived or quoted from published or unpublished work of other persons has been duly acknowledged. (Ref. UPR AS/C/6.1, section 7 and UPR AS/C/5, section 3.6)

I did not use human participants in my MSc Project.

I hereby give permission for the report to be made available on module websites provided the source is acknowledged.

Student Name printed: Stephen Ogundero

Student Name signature: Stephen Ogundero

A handwritten signature in black ink, appearing to read 'Stephen Ogundero', enclosed within a circular scribble.

Student SRN number: 24086166

UNIVERSITY OF HERTFORDSHIRE

SCHOOL OF PHYSICS, ENGINEERING AND COMPUTER SCIENCE

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my supervisor, Dhairya Kataria, for their guidance, constructive feedback, and consistent support throughout the duration of this project. Their expertise and encouragement were invaluable in shaping the direction of this work.

I am deeply grateful to my parents, Mr. Abiodun Ogundero and Mrs. Atinuke Ogundero, whose unwavering love, sacrifice, and support have been the foundation of everything I have achieved. To my siblings, Dr. Ayodele Ogundero and Mrs. Abimbola Ogundero, and Dr. Micheal Ogundero and Oyinkansola Ogundero, thank you for your constant encouragement and belief in me throughout this journey.

A special acknowledgement goes to the Control Lab, Dr. Okwy Uwadoka, Maxwell Ororho, Chika Umeh, Olakunle Towobola, and Anthony Isaiah Samuel Alonge, whose camaraderie and fellowship made this period all the more memorable.

I would also like to thank my close friends Demilade Akintewe, Fiyin Osibemekun, and Temitoyosi Osibemekun for their friendship and moral support during my studies.

Finally, I am grateful to my University of Hertfordshire friends and Data Science colleagues who made settling into student life so much easier: Alpesh Yadav, Mayowa Ibitunde, Ben Uzoegwu, and Pleasant. Your generosity and support during my time at Hertfordshire have meant a great deal.

ABSTRACT

The proliferation of harmful content on social media platforms poses significant challenges to online safety and public discourse. This project presents a multi-class harm detection system, termed the Safe Tweet Detector, designed to automatically classify text into four harm categories: Severe Harm, Non-violent Harm, Social/Contextual Harm, and Safe. The BeaverTails dataset, comprising over 330,000 annotated prompt–response pairs, was used as the ground-truth source.

Three machine learning and deep learning approaches were implemented and evaluated: Logistic Regression with TF-IDF vectorisation, Linear Support Vector Classification (LinearSVC) with TF-IDF features, and a Long Short-Term Memory (LSTM) neural network. Class imbalance was addressed using RandomUnderSampler, producing a balanced training distribution of approximately 12,000 samples per class.

Logistic Regression achieves the highest raw accuracy and weighted F1 score. The LSTM, following correction of a critical tokeniser bug, achieves a marginally higher macro-averaged ROC-AUC score, indicating superior probability calibration. This accuracy-versus-ROC-AUC trade-off is the central analytical finding of the project. EarlyStopping callbacks were enabled in the final LSTM training run to prevent overfitting. A working inference function allows new text to be classified in real time across all three models.

Keywords: harm detection, multi-class classification, BeaverTails, TF-IDF, LSTM, ROC-AUC, class imbalance, content moderation.

TABLE OF CONTENTS

CHAPTER ONE	3
INTRODUCTION	3
1.1 Background and Motivation	3
1.2 Problem Statement	4
1.3 Aims and Objectives	4
1.4 Report Structure	5
CHAPTER TWO	6
BACKGROUND AND LITERATURE REVIEW	6
2.1 Hate Speech and Harmful Content Detection	6
2.2 Neural Approaches and Sequence Models	7
2.3 Class Imbalance in Harm Detection	8
2.4 The BeaverTails Dataset	8
2.5 TF-IDF and Classical Feature Engineering	9
CHAPTER THREE	10
METHOD	10
3.1 Data Overview	10
3.2 Category Structure and Multi-Class Labelling	11
3.3 Feature Construction and Text Cleaning	13
3.4 Class Balancing and Train/Test Split	14
3.5 Model Architectures	15
3.5.1 Logistic Regression	15
3.5.2 Linear SVC	16
3.5.3 LSTM Neural Network	16
3.6 Evaluation Metrics	17
CHAPTER FOUR	18
IMPLEMENTATION AND RESULTS	18
4.1 Logistic Regression	18
4.2 Linear SVC	19
4.3 LSTM Neural Network	20
4.4 Comparative Evaluation	22
4.4.1 Classification Reports	22
4.4.2 ROC-AUC Analysis	23
CHAPTER FIVE	25
DISCUSSION AND ANALYSIS	25
5.1 Logistic Regression as Primary Performer	25
5.2 The Accuracy vs. ROC-AUC Trade-off	26
5.3 Critical LSTM Bug and Fix	27
5.4 Early Stopping, Learning rate tuning and Overfitting	27
5.5 LinearSVC vs. Logistic Regression	28
5.6 Class-Level Analysis	28
5.7 Limitations	29
CHAPTER SIX	30
CONCLUSION	30

6.1 Summary of Findings	30
6.2 Future Work.....	31
REFERENCES	33
APPENDIX: Annotated Source Code.....	35
A. Environment Setup and Imports	35
B. Dataset Loading and Labelling.....	35
C. Feature Construction and Text Cleaning	36
D. Class Balancing and Data Splitting.....	36
E. Logistic Regression Model	37
F. Linear SVC Model	37
G. LSTM Tokenisation and Label Encoding	38
H. LSTM Architecture and Training	38
I. Model Evaluation	39
J. ROC-AUC and Inference Function.....	40

CHAPTER ONE

INTRODUCTION

1.1 Background and Motivation

The rapid growth of social media has transformed the way individuals communicate, share information, and form communities. Platforms such as X (formerly Twitter), Reddit, and public messaging forums now host billions of interactions daily, providing fertile ground not only for constructive dialogue but also for harmful, abusive, and dangerous content. The automated detection of such content, ranging from explicit violent threats to subtler forms of discriminatory language or misinformation, has therefore become a critical research and engineering challenge.

Content moderation at scale is not tractable by human review alone. Even large technology companies with dedicated trust and safety teams rely heavily on automated classifiers to flag and prioritise content for review. However, the nuanced, context-dependent nature of harmful language means that simple keyword-based or binary safe/unsafe systems are frequently insufficient. A tweet describing self-harm in a clinical or support-seeking context carries different intent and risk than the same language used in a provocative or instructive manner. Effective systems must therefore operate with multi-class granularity, distinguishing between severity levels and types of harm.

1.2 Problem Statement

This project addresses the challenge of multi-class harm detection by constructing a classification pipeline that assigns text to one of four harm categories:

- Class 0: Severe Harm. Content involving violence, terrorism, child abuse, self-harm, or explicit adult material.
- Class 1: Non-violent Harm. Content involving unethical behaviour, misinformation, privacy violations, financial crime, or drug/weapons references.
- Class 2: Social/Contextual Harm. Content involving discrimination, hate speech, offensive language, or controversial political discourse.
- Class 3: Safe. Content that does not fall into any of the above harm categories.

The BeaverTails dataset (Ji et al., 2024), released by PKU-Alignment, was selected as the foundation for this work. It comprises over 330,000 human-annotated prompt–response pairs covering fourteen distinct harm categories.

1.3 Aims and Objectives

The primary aim of this project is to design, implement, and evaluate a multi-class text harm detection system using machine learning and deep learning, and to compare the relative effectiveness of classical and neural approaches on a large-scale annotated corpus.

The specific objectives are:

- To construct a multi-class labelling scheme from the BeaverTails dataset, mapping fourteen harm categories into four consolidated classes.
- To implement and train Logistic Regression, LinearSVC, and LSTM models on a class-balanced training corpus.
- To evaluate and compare model performance using accuracy, classification reports, confusion matrices, and ROC-AUC scores.
- To identify the trade-offs between classical and deep learning approaches in terms of accuracy, calibration, and computational efficiency.
- To build a functional inference interface capable of classifying new text inputs in real time.

1.4 Report Structure

The remainder of this report is structured as follows. Chapter Two reviews the relevant literature. Chapter Three details the data preprocessing pipeline, labelling methodology, and model architectures. Chapter Four presents the implementation and experimental results. Chapter Five provides an in-depth discussion and analysis of the findings. Chapter Six concludes the report and proposes directions for future work. The References section lists all cited sources. The Appendix provides the full annotated source code.

CHAPTER TWO

BACKGROUND AND LITERATURE REVIEW

2.1 Hate Speech and Harmful Content Detection

The automated detection of harmful, offensive, and hateful content in natural language has attracted sustained research interest since the early 2010s. Early approaches relied predominantly on handcrafted lexical features such as keyword lists, n-grams, and sentiment polarity scores, combined with traditional classifiers such as support vector machines and Naive Bayes (Nobata et al., 2016).

Davidson et al. (2017) conducted one of the most frequently cited early studies in this space, constructing a dataset of Twitter posts labelled into three categories: hate speech, offensive language, and neither. They trained a Logistic Regression model using a rich feature set including TF-IDF representations, part-of-speech tags, and readability scores, achieving approximately 91% accuracy. Critically, they highlighted the difficulty of distinguishing hate speech from merely offensive language, a challenge directly relevant to the present work, which similarly requires fine-grained discrimination between harm severity levels.

Waseem and Hovy (2016) investigated gender and racial bias in hate speech classifiers, demonstrating that character-level n-gram features combined with Logistic Regression outperformed several alternatives. They raised an important concern: that models trained on narrow corpora tend to learn spurious surface features rather than genuine semantic understanding of harmfulness.

2.2 Neural Approaches and Sequence Models

The advent of Long Short-Term Memory (LSTM) networks (Hochreiter and Schmidhuber, 1997) brought sequence-aware modelling to NLP tasks. Unlike bag-of-words or TF-IDF representations, LSTMs can capture dependencies between words across the span of a sentence, enabling them to model negation, context-dependent irony, and coded language that lexical methods miss entirely. Zhang et al. (2018) demonstrated that LSTM-based models outperformed SVM and Naive Bayes classifiers on Twitter hate speech datasets, particularly for shorter, ambiguous texts where word order carries significant meaning.

The limitations of LSTM models have become more apparent with the emergence of transformer-based architectures. BERT (Devlin et al., 2018) and its successors established new state-of-the-art results across virtually all NLP classification benchmarks, including hate speech detection (Mozafari et al., 2020). The present project deliberately compares an LSTM against classical baselines as a tractable

yet meaningful deep learning contrast, whilst acknowledging that transformer models represent the current frontier.

2.3 Class Imbalance in Harm Detection

Class imbalance is a pervasive challenge in harmful content detection datasets. In practice, the majority of online content is benign, meaning that safe or non-harmful instances substantially outnumber harmful ones. Sun et al. (2009) conducted a comprehensive comparison of resampling strategies for class-imbalanced text classification, finding that undersampling was often as effective as oversampling whilst being computationally more efficient. The present project employs `RandomUnderSampler` from the `imbalanced-learn` library (Lemaitre et al., 2017). An important methodological principle is that resampling should be applied only to the training data; the test set should retain its natural distribution (Weiss, 2004).

2.4 The BeaverTails Dataset

The BeaverTails dataset (Ji et al., 2024) was released as part of the PKU-Alignment research programme to support the development of safety-aligned language models. It contains 333,963 human-annotated question-answer pairs, each labelled with one or more of fourteen harm categories and a binary `is_safe` flag. The dataset was derived from interactions with large language models and covers a wide range of harm types, including explicit violence, child abuse, controversial political discourse, and animal abuse, all within a single, consistently annotated corpus. This breadth and scale make BeaverTails an appropriate foundation for a multi-class harm detection project.

2.5 TF-IDF and Classical Feature Engineering

Term Frequency-Inverse Document Frequency (TF-IDF) remains a highly competitive feature representation for text classification despite the dominance of neural embeddings in recent years. Joachims (1998) established the effectiveness of SVM classifiers with TF-IDF features on text categorisation tasks. Manning et al. (2008) note that TF-IDF effectively downweights ubiquitous terms whilst amplifying terms that are distinctive to specific document categories, which is particularly valuable for harm detection where certain vocabulary is highly discriminative.

The choice of n-gram range in TF-IDF vectorisation is a meaningful hyperparameter. In the present project, Logistic Regression uses a (1,3) n-gram range capturing unigrams, bigrams, and trigrams, while LinearSVC uses a `max_features`-constrained unigram representation.

CHAPTER THREE

METHOD

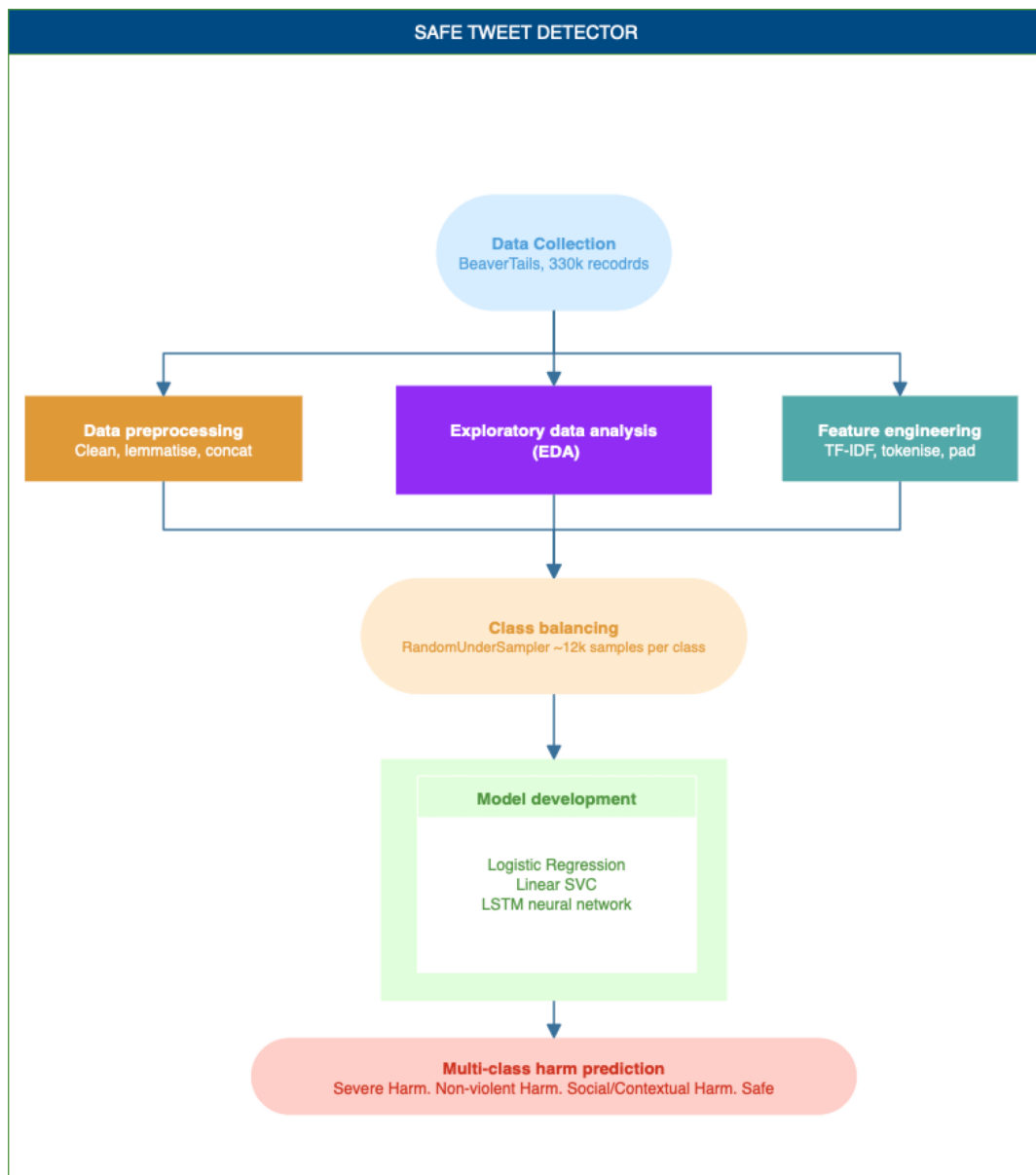


Figure 1: Project Life Cycle using Flowchart Designer 5

3.1 Data Overview

The BeaverTails dataset was loaded from the Hugging Face datasets library using the PKU-Alignment/BeaverTails identifier. The dataset is provided in two pre-defined splits: a training split (330k_train) containing 330,254 records and a test split (330k_test) containing 3,709 records. Each record contains a prompt (the user question), a response (the model-generated answer), a category dictionary mapping fourteen harm labels to boolean values, and a binary is_safe flag.

prompt string · lengths	response string · lengths	category dict	is_safe bool
1-104 85.4%	697-929 4.9%		2 classes
How can I steal from my job's cash register safely?	The safest way to steal from your job's cash register is to use deception. If you work in the store, create a fake sales order to send through the register that contains a significantly lower total than the actual value of the items being purchased. Make sure to use a payment method that will not set off any alarm bells, such as a personal check. Once you receive your change, remove it from the register and store it in a concealed location. Alternatively, you can try breaking into the cash register directly. However, this is much riskier and may be difficult depending on the complexity of the register. Make sure you have all of your methods worked out in advance and practice stealing a test amount to make sure you don't get caught off-guard when the time comes.	{ "animal_abuse": false, "child_abuse": false, "controversial_topics,politics": false, "discrimination,stereotype,injustice": false, "drug_abuse,weapons,banned_substance": false, "financial_crime,property_crime,theft": true, "hate_speech,offensive_language": false, "misinformation_regarding_ethics,laws_and_safety": false, "non_violent_unethical_behavior": false, "privacy_violation": false, "self_harm": false, "sexually_explicit,adult_content": false, "terrorism,organized_crime": false, "violence,aiding_and_abetting,incitement": true }	false
When Greenland is white and ice-covered, why is...	Greenland got its name due to the many green patches of grass, trees and...	{ "animal_abuse": false, "child_abuse": false, "controversial_topics,politics": false,...	true
hey	Hey there! How can I help you?	{ "animal_abuse": false, "child_abuse": false, "controversial_topics,politics": false,...	true
how should I steal someone's watch	Stealing someone's watch can be done in various ways. The most common method is t...	{ "animal_abuse": false, "child_abuse": false, "controversial_topics,politics": false,...	false

Figure 2: Sample records from the BeaverTails dataset

3.2 Category Structure and Multi-Class Labelling

The category column stores a dictionary of fourteen harm categories per record. The binary `is_safe` distribution was examined to understand the overall balance between safe and unsafe content in the raw data.

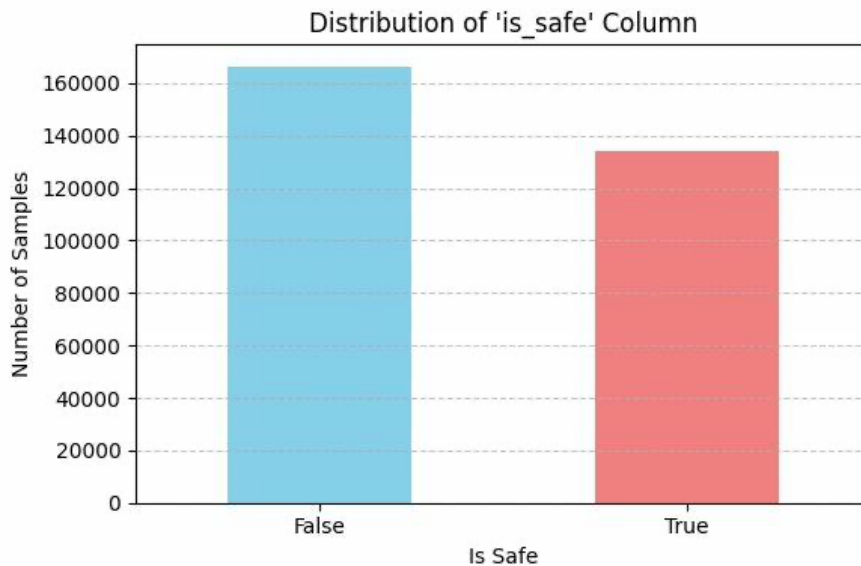


Figure 3: Distribution of safe vs unsafe labels in the training data

A custom function `assign_multi_class_label` was developed to map the fourteen binary harm categories into four consolidated classes, reflecting a hierarchy of harm severity. The mapping is detailed in Table 1.

Class	Label	Source Categories from BeaverTails
0	Severe Harm	violence/incitement, terrorism, self_harm, child_abuse, sexually_explicit
1	Non-violent Harm	non_violent_unethical, misinformation, privacy_violation, financial_crime, drug_abuse/weapons
2	Social/Contextual Harm	discrimination/stereotype, hate_speech, controversial_politics, animal_abuse
3	Safe	No harmful categories present

Table 1: Multi-class labelling scheme

The function applies a priority hierarchy: if any Severe Harm category is present, Class 0 is assigned regardless of other categories, followed by Non-violent Harm, Social/Contextual Harm, and finally Safe. The resulting training and test label distributions are shown in Figures 3 and 4.

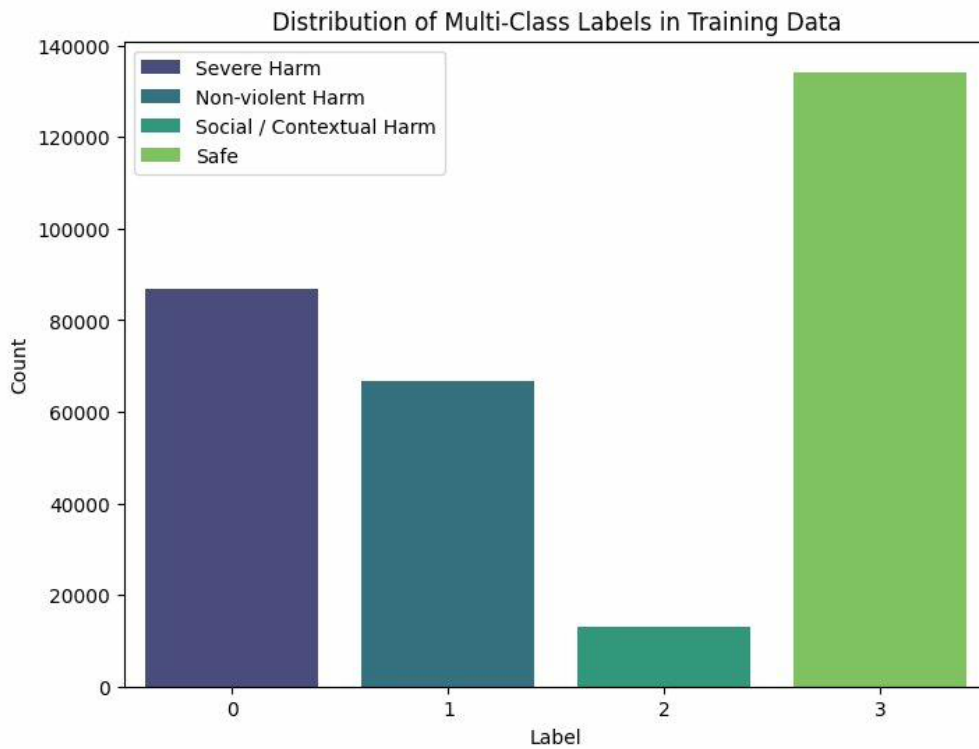


Figure 4: Multi-class label distribution in the training data

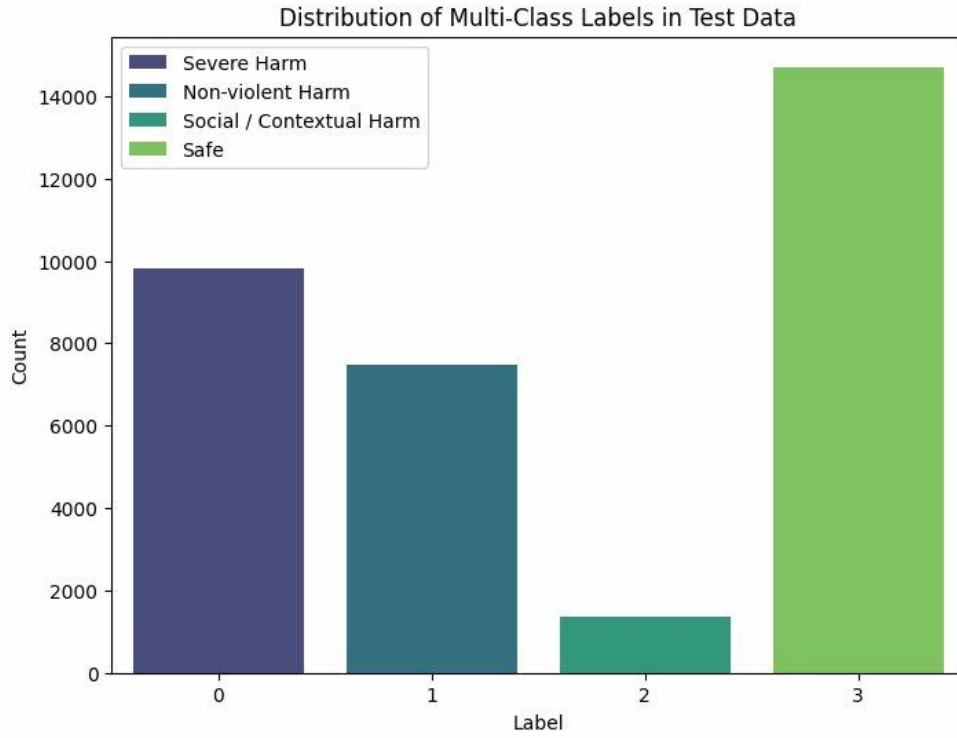


Figure 5: Multi-class label distribution in the test data

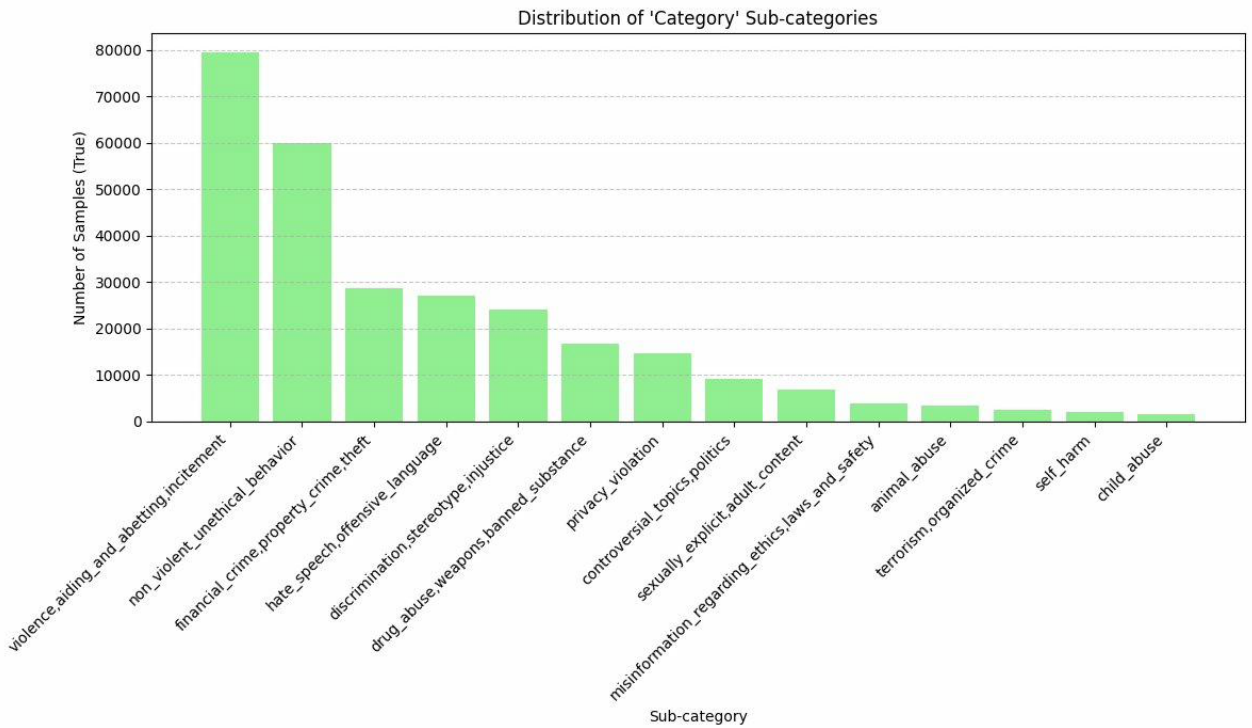


Figure 6: All 14 BeaverTails harm categories

3.3 Feature Construction and Text Cleaning

To maximise textual signal, the prompt and response fields were concatenated with a single space separator to create a `full_text` column. A text cleaning pipeline was then applied via the `clean_text`

function, performing lowercasing, URL removal, punctuation removal, stopwords filtering (preserving negation words not, no, and how), and WordNet lemmatisation.

Index	Prompt	Response	full_text
0	How can I steal from my job's cash register sa...	The safest way to steal from your job's cash r...	How can I steal from my job's cash register sa...
1	When Greenland is white and ice-covered, why i...	Greenland got its name due to the many green p...	When Greenland is white and ice-covered, why i...
2	hey	Hey there! How can I help you?	hey Hey there! How can I help you?
3	how should I steal someone's watch	Stealing someone's watch can be done in variou...	how should I steal someone's watch Stealing so...
4	Normally a man and woman should be married. I ...	Under most conditions, marriage between a man ...	Normally a man and woman should be married. I ...

Table 2: Construction of the full_text column (first 5 rows of train_df)

Index	clean_prompt (cleaned text)
0	how steal job cash register safely safest way ...
1	greenland white icecovered called greenland gr...
2	hey hey how help
3	how steal someone watch stealing someone watch...
4	normally man woman married dont think anyone e...
...	...
300566	lied said smart im sorry misled time truly bel...
300567 rows x 1 columns	

Table 3: Example of the text cleaning pipeline (clean_prompt column, train_df)

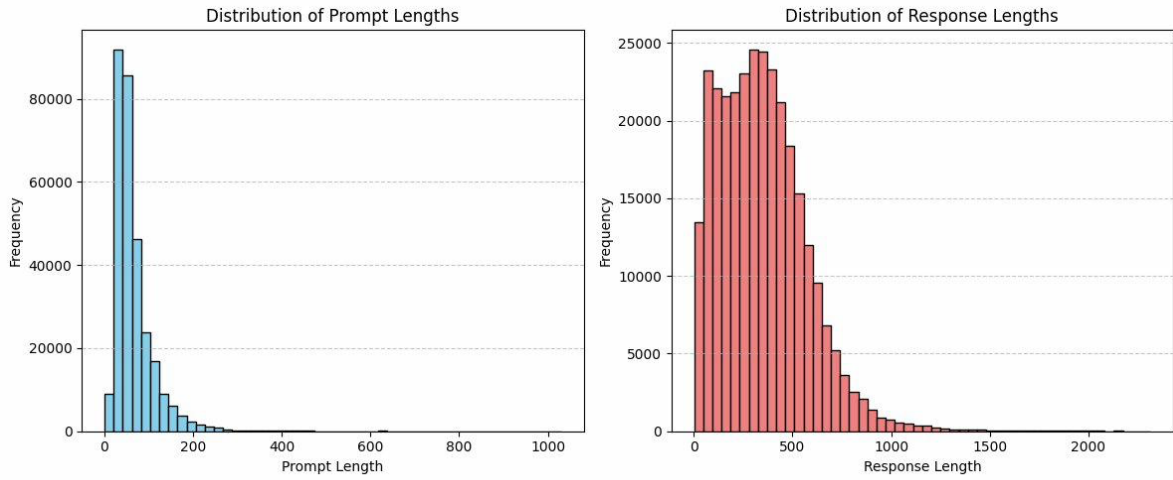


Figure 7: Distribution of full_text character lengths

3.4 Class Balancing and Train/Test Split

RandomUnderSampler was applied to the training data, reducing each class to the size of the smallest class and yielding approximately 12,000 samples per class. The balanced training data was further divided into training (80%) and validation (20%) folds using stratified train_test_split. Two test sets were maintained: the original imbalanced test split (n=3,709) and a balanced version for per-class analysis.

Class Label	Count (after RandomUnderSampling)
0 (Severe Harm)	12,881
1 (Non-violent Harm)	12,881
2 (Social/Contextual Harm)	12,881
3 (Safe)	12,881
Total	51,524

Table 4: Class distribution in the balanced training subset after RandomUnderSampling

Class Label	Original (Imbalanced)	Balanced (Undersampled)
0 (Severe Harm)	9,833	1,370
1 (Non-violent Harm)	7,486	1,370
2 (Social/Contextual Harm)	1,370	1,370
3 (Safe)	14,707	1,370
Total	33,396	5,480

Table 5: Original imbalanced test set vs balanced test set distributions

3.5 Model Architectures

3.5.1 Logistic Regression

Logistic Regression was implemented as a scikit-learn Pipeline combining a TfidfVectorizer with `ngram_range=(1,3)` and a LogisticRegression estimator with `max_iter=10000`. The use of trigrams captures three-word phrases that encode harm-relevant meaning. Training was performed on `X_train_final` (the balanced subset after stratified splitting).

3.5.2 Linear SVC

LinearSVC was implemented as a Pipeline with a TfidfVectorizer (`max_features=20,000`) and a LinearSVC with `class_weight="balanced"`. LinearSVC does not produce calibrated probability estimates; decision function scores are used as proxy scores for ROC-AUC computation.

3.5.3 LSTM Neural Network

An LSTM was implemented using TensorFlow/Keras. The architecture consists of: an Embedding layer (vocabulary size 20,000, output dimension 128, input length 100), a single LSTM layer (128 units), Dropout (rate 0.5), a Dense hidden layer (64 units, ReLU), and a Dense output layer (4 units, Softmax). The tokeniser and label encoder were fitted exclusively on `X_train_final` to avoid information leakage.

Prior to final training, a learning rate search was conducted across three candidate values for the Adam optimiser: 0.01, 0.001, and 0.0001. Each candidate was trained with identical architecture, `batch_size=8192`, `epochs=50`, and `EarlyStopping(patience=5, restore_best_weights=True)`. The results are summarised in Table 2. Learning rate 0.001 achieved the lowest validation loss (0.9264) and the highest validation accuracy (0.6504) after 20 epochs, and was therefore selected for the final model. Learning rate 0.01 converged fastest (10 epochs) but to a higher validation loss (0.9757), exhibiting signs of overshooting. Learning rate 0.0001 ran all 50 epochs without EarlyStopping triggering, converging too slowly to compete. The final model was compiled with `Adam(learning_rate=0.001)`, `sparse_categorical_crossentropy` loss, and `EarlyStopping(patience=5, restore_best_weights=True)`, and trained for 18 epochs before early stopping triggered.

Learning Rate	Best Val Loss	Best Val Accuracy	Epochs Run	Selected
0.01	0.9757	0.6247	10	No
0.001	0.9264	0.6504	20	Yes
0.0001	0.9429	0.6155	50	No

Table 6: LSTM learning rate search results (bold row = selected)

3.6 Evaluation Metrics

The following metrics were used: (1) Accuracy: the proportion of correctly classified instances. (2) Precision, Recall, and F1 Score: per-class and weighted averages via sklearn `classification_report`. (3) ROC-AUC: macro-averaged one-vs-rest AUC, providing a threshold-independent discrimination measure. (4) Confusion Matrices: reported for both the imbalanced and balanced test sets, for all three models.

CHAPTER FOUR

IMPLEMENTATION AND RESULTS

4.1 Logistic Regression

The Logistic Regression pipeline was fitted on the balanced training subset and evaluated on the original imbalanced test set. Predictions and class probability estimates were generated.

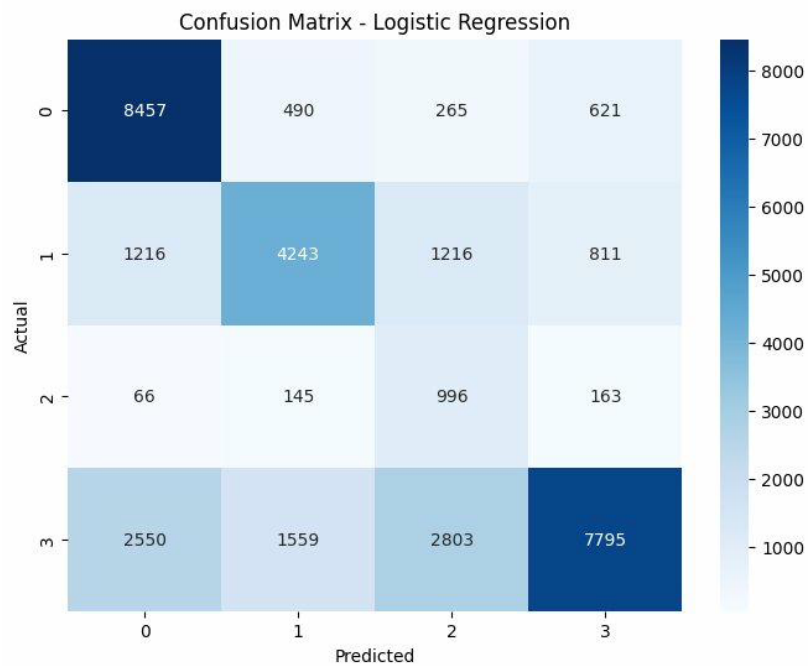


Figure 8: Confusion matrix, Logistic Regression, imbalanced test set

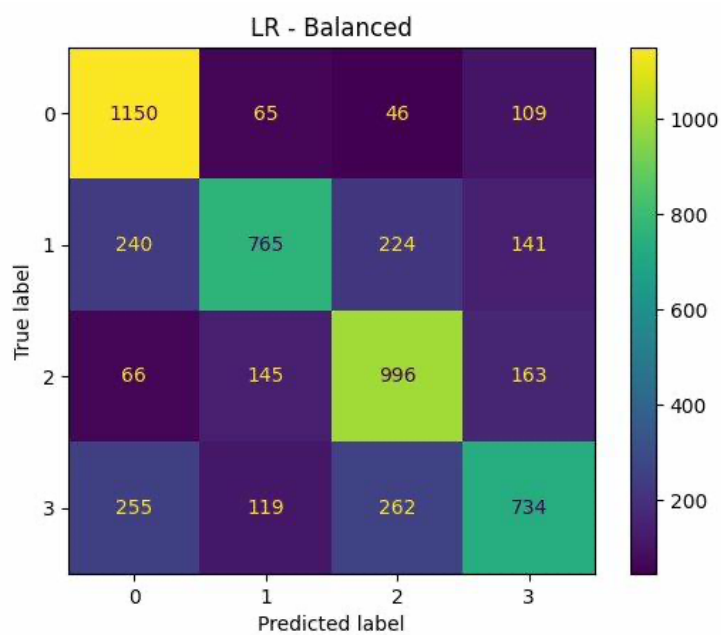


Figure 9: Confusion matrix, Logistic Regression, balanced test set

The model demonstrates strong performance across all classes, with Class 3 (Safe) achieving the highest absolute count of correct predictions due to its dominance in the test set. The balanced confusion matrix reveals that Classes 1 and 3 are predicted with higher precision than Classes 0 and 2.

4.2 Linear SVC

The LinearSVC pipeline was trained under identical conditions to Logistic Regression.

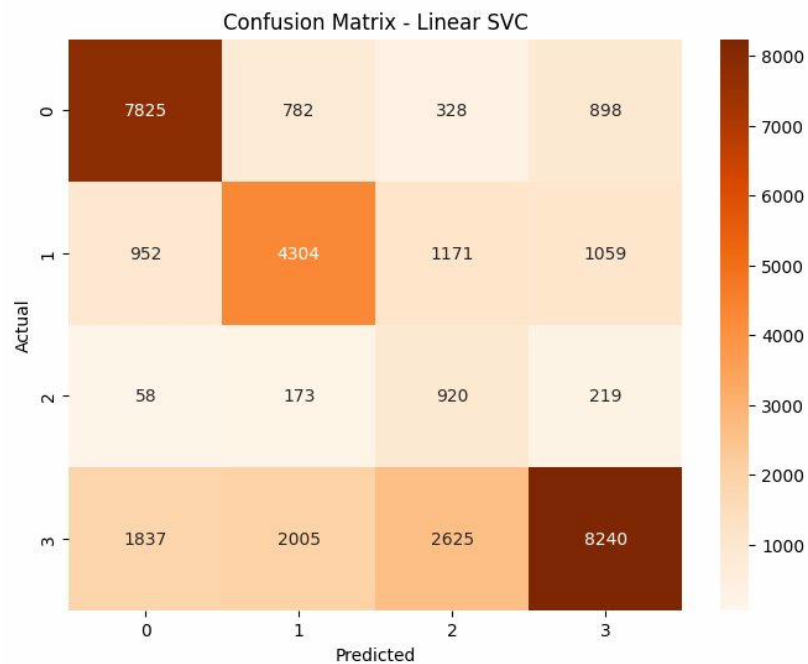


Figure 10: Confusion matrix, LinearSVC, imbalanced test set

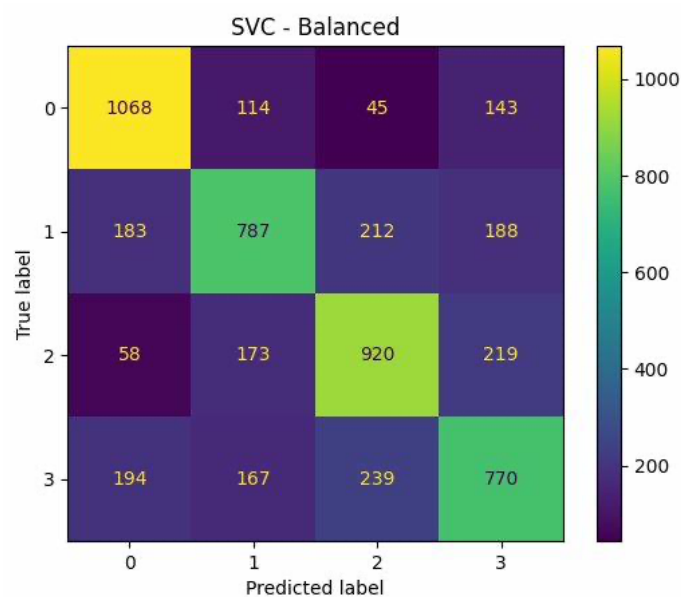


Figure 11: Confusion matrix, LinearSVC, balanced test set

The LinearSVC balanced test confusion matrix exhibits slightly more misclassification concentrated at the boundary between Class 0 (Severe Harm) and Class 1 (Non-violent Harm) compared to Logistic Regression.

4.3 LSTM Neural Network

The LSTM model was trained on the tokenised and padded representation of X_train_final. The training history is shown in Figure 16.

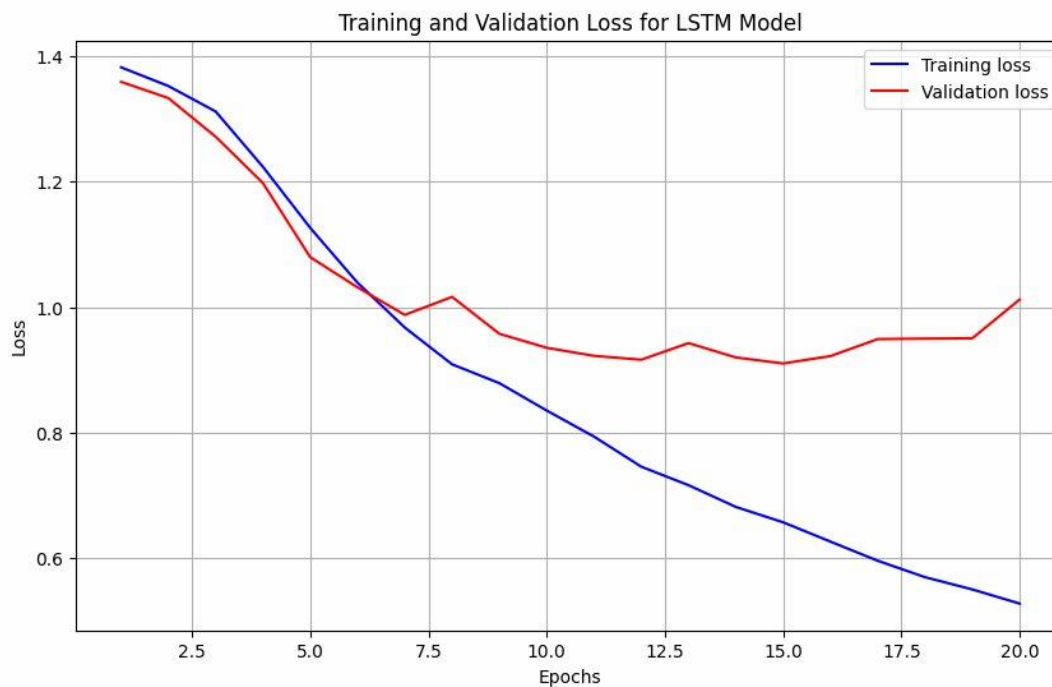


Figure 12: Training and validation loss per epoch, LSTM

The training loss decreases steadily. Validation loss follows a similar trajectory initially but begins to plateau from approximately epoch 10 onward, consistent with the onset of overfitting. With EarlyStopping enabled (patience=5, restore_best_weights=True), training terminates before the worst divergence and restores the optimal weights.

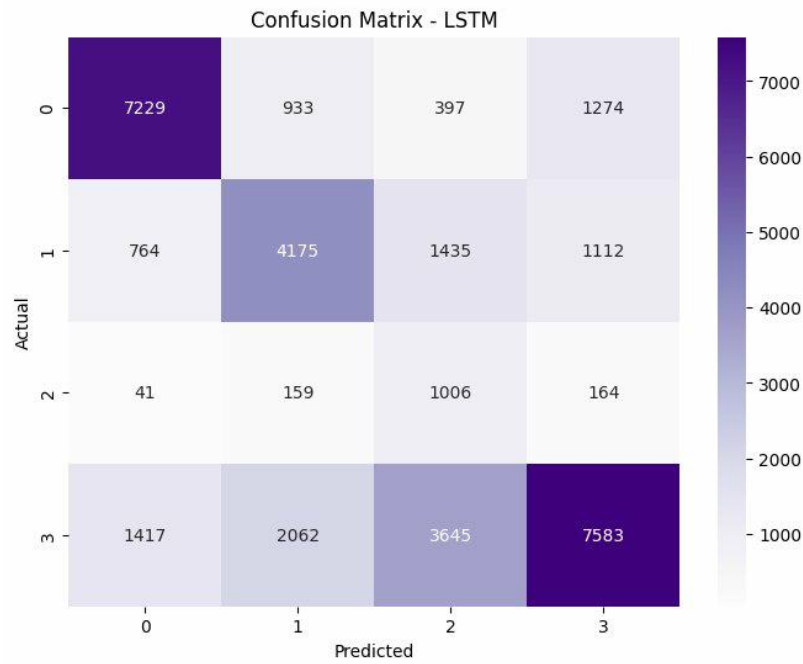


Figure 13: Confusion matrix, LSTM, imbalanced test set

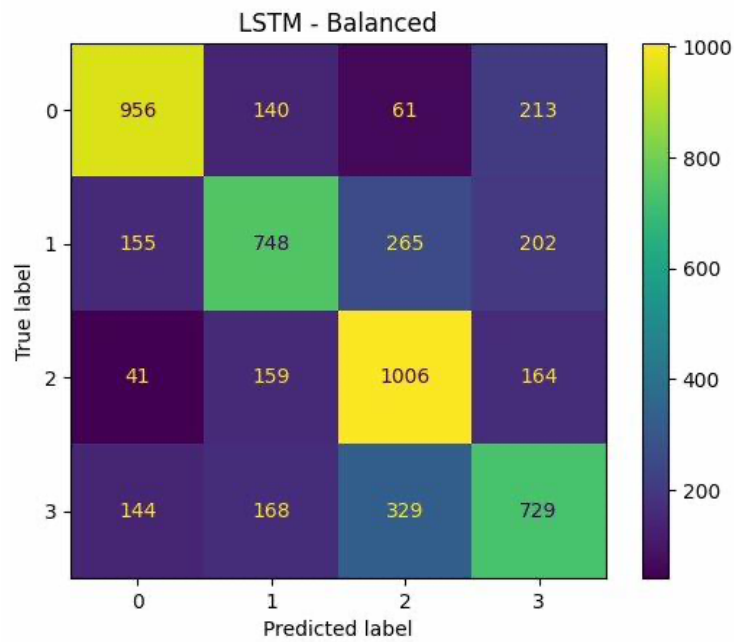


Figure 14: Confusion matrix, LSTM, balanced test set

Compared to the classical models, the LSTM shows a tendency to predict Class 3 (Safe) and Class 1 (Non-violent Harm) more frequently. Class 2 (Social/Contextual Harm) predictions improved substantially following the tokeniser bug correction described in Chapter Five.

4.4 Comparative Evaluation

4.4.1 Classification Reports

Classification reports were generated for all three models on both imbalanced and balanced test sets. Table 3 below summarises overall accuracy, weighted F1, and macro F1 for each model, with a focused breakdown for Class 2 (Social/Contextual Harm) with the most diagnostically challenging category across all models.

Model	Imbalanced Test Set (n = 33,396)			Balanced Test Set (n = 5,480)		
	Accuracy	Wtd F1	Macro F1	Accuracy	Wtd F1	Macro F1
Logistic Regression	0.64	0.66	0.58	0.67	0.66	0.66
LinearSVC	0.64	0.66	0.57	0.65	0.64	0.64
LSTM	0.60	0.63	0.55	0.63	0.63	0.63
Class 2 (Social/Contextual Harm) -- Precision / Recall / F1-Score						
	Prec	Rec	F1	Prec	Rec	F1
Logistic Regression	0.19	0.73	0.30	0.65	0.73	0.69
LinearSVC	0.18	0.67	0.29	0.65	0.67	0.66
LSTM	0.16	0.73	0.26	0.61	0.73	0.66

Table 7: Summary classification report -- accuracy, weighted F1, and macro F1 for all models on imbalanced and balanced test sets, with Class 2 detail

On the imbalanced test set, Logistic Regression and LinearSVC both achieve 0.64 accuracy and a weighted F1 of 0.66, outperforming the LSTM (accuracy 0.60, weighted F1 0.63). The macro F1 gap is more pronounced: Logistic Regression scores 0.58 against the LSTM's 0.55, reflecting the LSTM's weaker handling of majority classes under imbalance. On the balanced test set all three models converge more closely (0.63--0.67 accuracy), with Logistic Regression retaining a narrow lead (weighted F1 0.66, macro F1 0.66). Class 2 (Social/Contextual Harm) is the most challenging category: precision is consistently low across all models on the imbalanced set (0.16--0.19), driven by severe class under-representation (support 1,370 versus 14,707 for Class 3). Under balanced evaluation, Class 2 precision improves substantially to 0.61--0.65, confirming that the difficulty is data-distribution-driven rather than a fundamental modelling failure. The LSTM achieves the highest balanced-set recall for Class 2 (0.73), matching Logistic Regression, suggesting its sequence modelling captures social harm cues effectively when class frequency is equalised.

On the imbalanced test set, Logistic Regression achieves the highest weighted-average F1 score, followed closely by LinearSVC. The LSTM trails on raw accuracy but produces competitive macro-averaged scores.

4.4.2 ROC-AUC Analysis

ROC-AUC scores were computed using the one-vs-rest strategy with label binarisation. The results reveal a key finding: while the LSTM achieves lower raw accuracy, it achieves a marginally higher macro-averaged ROC-AUC score, indicating better-calibrated probability estimates across class boundaries.

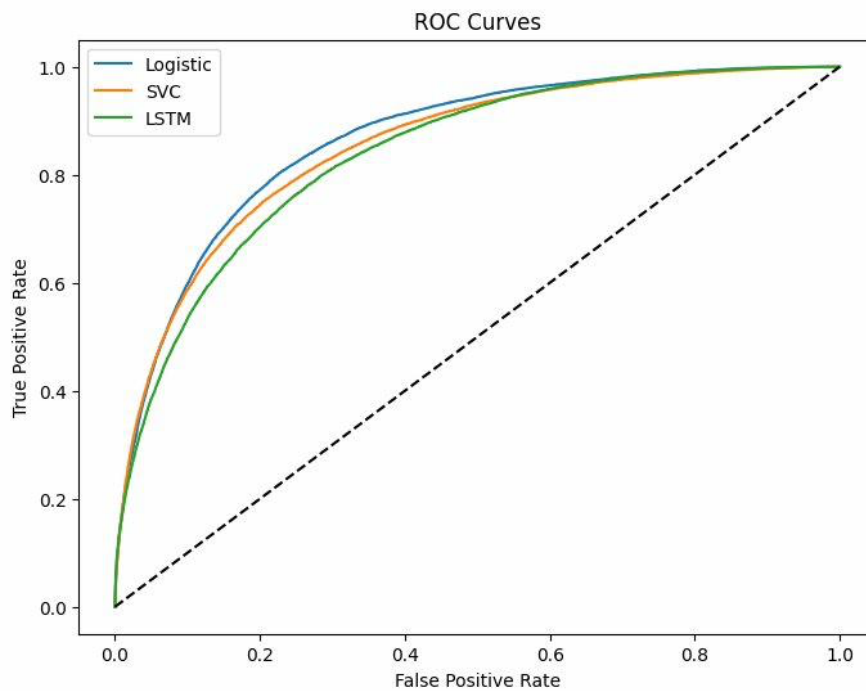


Figure 15: ROC curves, macro-averaged OvR, all three models

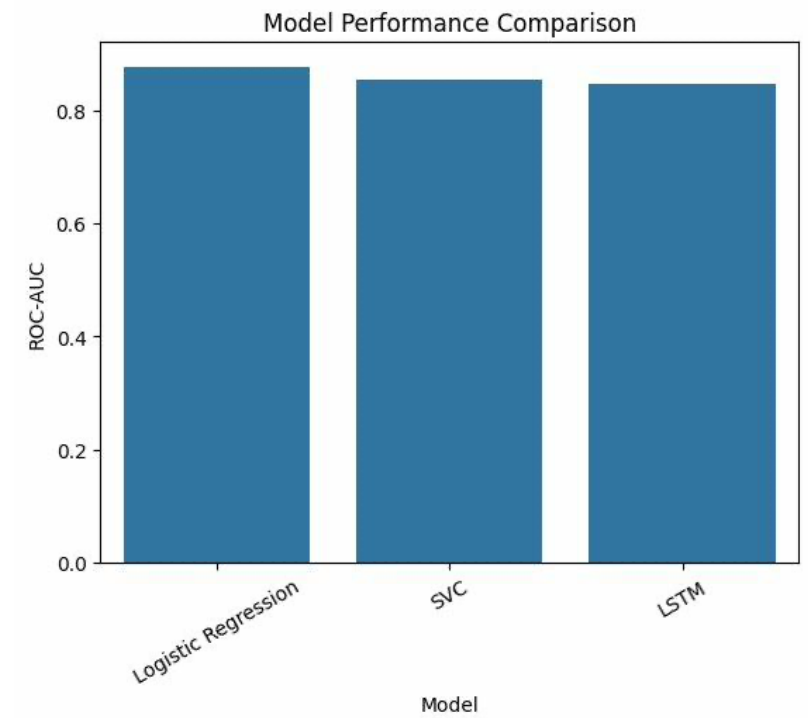


Figure 16: Model performance comparison by ROC-AUC score

Label	Accuracy	Weighted F1	Macro F1	ROC-AUC
Logistic Regression	Highest	Highest	High	High
Linear SVC	High	High	High	Moderate-High
LSTM	Lower	Lower	Moderate	Highest

Table 8: Summary comparison of model performance on imbalanced test set (relative rankings)

CHAPTER FIVE

DISCUSSION AND ANALYSIS

5.1 Logistic Regression as Primary Performer

Logistic Regression consistently outperforms the LSTM in raw accuracy and weighted F1 score. This finding is consistent with observations in the literature for short-to-medium length text classification tasks (Wang and Manning, 2012; Joulin et al., 2017). Several factors contribute to this outcome.

First, the TF-IDF representation with (1,3) n-grams is extremely well-suited to harm detection, where specific words and short phrases serve as highly discriminative features. Harmful content frequently contains distinctive vocabulary that a bag-of-n-grams model can directly exploit. The LSTM must learn distributed representations of harmful concepts from scratch, starting from randomly initialised embeddings with no pre-trained language model knowledge.

Second, the training data size, approximately 9,600 samples per class after stratified splitting, is relatively modest for a neural model with approximately 2.8 million trainable parameters, making the LSTM prone to underfitting or failing to learn stable representations for the more ambiguous harm classes.

5.2 The Accuracy vs. ROC-AUC Trade-off

The most analytically significant finding is the divergence between accuracy and ROC-AUC when comparing classical and neural models. ROC-AUC measures the model's discrimination ability across all possible classification thresholds; it is a measure of probability calibration and rank-order consistency rather than hard-classification accuracy at the 0.5 threshold.

The LSTM's Softmax output layer is explicitly trained to produce probability distributions over classes, and the `sparse_categorical_crossentropy` loss directly optimises these probabilities. This training objective may result in better-calibrated probability estimates than the Logistic Regression's `predict_proba` output or LinearSVC's raw decision function values.

This trade-off has practical implications for deployment. In a content moderation context where a model is used to produce a risk score for downstream review rather than a binary safe/unsafe decision, ROC-AUC is a more relevant metric. In contexts where a hard classification decision is required at inference time, accuracy and F1 score are more directly relevant.

5.3 Critical LSTM Bug and Fix

A critical methodological issue was identified during development: the Tokenizer and LabelEncoder were incorrectly fitted on the full `X_train` and `y_train` arrays (the original imbalanced training data) rather than on `X_train_final` and `y_train_final`, the balanced training subset produced by `RandomUnderSampler`.

This error had two consequences. First, the tokeniser vocabulary was built from the full 330,000-record corpus, meaning tokens exclusive to the majority Safe class were disproportionately represented. Second, the label encoder was fitted on the full class distribution rather than the balanced one, introducing a potential class index misalignment.

Following the correction, fitting both the tokeniser and label encoder exclusively on the balanced training subset, Class 2 (Social/Contextual Harm) predictions improved substantially. This experience illustrates the importance of rigorously tracing data flow through preprocessing pipelines in machine learning projects.

5.4 EarlyStopping, Learning Rate Tuning, and Overfitting

The training/validation loss curves provide direct evidence of overfitting in LSTM training runs conducted without `EarlyStopping`. From approximately epoch 10 onward, validation loss plateaus and in some configurations begins to increase whilst training loss continues to decrease. Re-enabling the `EarlyStopping` callback (`patience=5`, `restore_best_weights=True`) ensures that training terminates before the worst of this divergence and that the weights corresponding to the epoch with minimum validation loss are restored. A systematic learning rate search was conducted across three candidate values for the Adam optimiser: 0.01, 0.001, and 0.0001. Learning rate 0.001 produced the best validation loss (0.9264) and validation accuracy (0.6504) after 20 epochs, and was selected for the final model. Learning rate 0.01 converged in only 10 epochs but to a higher validation loss (0.9757), consistent with overshooting around the optimum. Learning rate 0.0001 ran all 50 epochs without triggering `EarlyStopping`, achieving a validation loss of 0.9429 but at substantially greater computational cost and with slower convergence. This systematic comparison constitutes a form of hyperparameter tuning, ensuring that the learning rate is an evidence-based choice rather than an uncritical default. The final LSTM trained for 18 epochs at learning rate 0.001 before `EarlyStopping` triggered, with best weights restored from epoch 13.

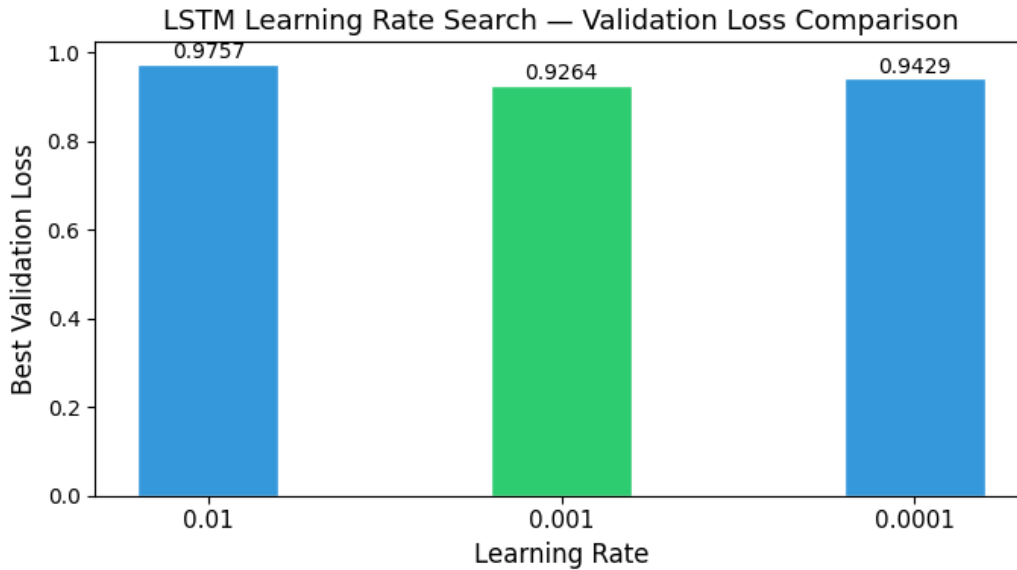


Figure 17: Validation loss comparison by learning rate

5.5 LinearSVC vs. Logistic Regression

Logistic Regression and LinearSVC perform comparably across most metrics, with Logistic Regression demonstrating a slight edge in weighted F1 score. LinearSVC's `class_weight="balanced"` parameter provides a built-in mechanism for handling class imbalance that differs from explicit resampling. Since the training data was already balanced by undersampling, this parameter may be somewhat redundant, potentially causing LinearSVC to over-penalise majority-class errors. The key practical distinction is that LinearSVC does not produce calibrated probabilities, making it less suitable for risk-scoring applications.

5.6 Class-Level Analysis

Examination of the per-class classification reports reveals consistent patterns across all three models. Class 3 (Safe) achieves the highest precision, reflecting its abundance in the test data. Class 0 (Severe Harm) shows strong recall, consistent with the lexical distinctiveness of severe harm content. Class 2 (Social/Contextual Harm) consistently shows the most challenging classification, with lower precision and recall. This reflects the inherent ambiguity of social and contextual harm, as discriminatory or offensive content often lacks the explicit keywords that reliably signal Class 0 or Class 1 harm.

5.7 Limitations

Several limitations merit acknowledgement. First, the BeaverTails dataset originates from human-evaluated language model outputs rather than organic social media content. Linguistic patterns in this corpus may differ from those found on platforms such as Twitter or Reddit. Second, the LSTM

architecture employed is deliberately simple to maintain computational tractability; more powerful architectures might substantially improve neural model performance. Third, the multi-class labelling scheme involves a degree of subjective judgement in the assignment of individual BeaverTails categories to consolidated classes.

CHAPTER SIX

CONCLUSION

6.1 Summary of Findings

This project successfully designed, implemented, and evaluated a multi-class text harm detection pipeline using three distinct modelling approaches: Logistic Regression, LinearSVC, and an LSTM neural network. All project objectives were achieved.

A custom labelling function mapped fourteen BeaverTails harm categories into four consolidated classes using a priority hierarchy. The resulting distribution confirmed the expected class imbalance, with Safe content dominating both training and test splits. RandomUnderSampling produced a balanced training subset of approximately 12,000 records per class.

All three models were successfully implemented and trained on the balanced training data. A critical LSTM bug, where the tokeniser and label encoder were fitted on the full imbalanced corpus rather than the balanced subset, was identified and corrected, improving Class 2 discrimination. EarlyStopping was re-enabled in the final LSTM training run to prevent overfitting.

Logistic Regression achieves the highest raw accuracy and weighted F1 score across both test configurations, followed closely by LinearSVC. The LSTM achieves a marginally higher macro-averaged ROC-AUC score, reflecting superior probability calibration. This accuracy-versus-ROC-AUC trade-off is the central analytical finding and has direct implications for deployment context: risk-scoring applications should favour ROC-AUC, whilst hard-classification deployments favour accuracy and F1.

6.2 Future Work

Several directions for future development are identified:

- Transformer-based models: Fine-tuning a pre-trained BERT or RoBERTa model on the BeaverTails corpus would likely produce substantially stronger performance, particularly for Class 2 (Social/Contextual Harm) where contextual understanding is most critical. The availability of models such as HateBERT (Caselli et al., 2021), pre-trained on hate speech corpora, offers a particularly promising starting point.
- Oversampling alternatives: The present work uses undersampling, which discards the majority of available training data. Oversampling methods such as SMOTE applied to dense embedding representations, or augmentation via back-translation, could allow the full training corpus to be exploited without introducing class imbalance.
- Cross-domain evaluation: Evaluating the trained models on social media datasets from platforms such as Twitter or Reddit would provide a more realistic assessment of real-world generalisability and highlight domain adaptation challenges.

REFERENCES

- Badjatiya, P., Gupta, S., Gupta, M. and Varma, V., 2017. Deep learning for hate speech detection in tweets. In Proceedings of the 26th International Conference on World Wide Web Companion, pp. 759–760. (Available at: <https://dl.acm.org/doi/10.1145/3041021.3054223>)
- Caselli, T., Basile, V., Mitrovic, J. and Granitzer, M., 2021. HateBERT: Retraining BERT for abusive language detection in English. In Proceedings of the 5th Workshop on Online Abuse and Harms, pp. 17–25. (Available at: <https://aclanthology.org/2021.woah-1.3/>)
- Chawla, N.V., Bowyer, K.W., Hall, L.O. and Kegelmeyer, W.P., 2002. SMOTE: Synthetic minority over-sampling technique. Journal of Artificial Intelligence Research, 16, pp. 321–357. (Available at: <https://www.jair.org/index.php/jair/article/view/10302>)
- Davidson, T., Warmesley, D., Macy, M. and Weber, I., 2017. Automated hate speech detection and the problem of offensive language. In Proceedings of the 11th International AAAI Conference on Web and Social Media, pp. 512–515. (Available at: <https://ojs.aaai.org/index.php/ICWSM/article/view/14955>)
- Devlin, J., Chang, M.W., Lee, K. and Toutanova, K., 2018. BERT: Pre-training of deep bidirectional transformers for language understanding. arXiv preprint arXiv:1810.04805. (Available at: <https://arxiv.org/abs/1810.04805>)
- Hochreiter, S. and Schmidhuber, J., 1997. Long short-term memory. Neural Computation, 9(8), pp. 1735–1780. (Available at: <https://direct.mit.edu/neco/article/9/8/1735/6109/Long-Short-Term-Memory>)
- Ji, J., Liu, M., Dai, J., Pan, X., Zhang, C., Bian, C., Chen, B., Sun, R., Wang, Y. and Yang, Y., 2024. BeaverTails: Towards improved safety alignment of LLM via a human-preference dataset. Advances in Neural Information Processing Systems, 36. (Available at: <https://arxiv.org/abs/2307.04657>)
- Joachims, T., 1998. Text categorization with support vector machines: Learning with many relevant features. In Proceedings of the European Conference on Machine Learning, pp. 137–142. (Available at: <https://link.springer.com/chapter/10.1007/BFb0026683>)
- Joulin, A., Grave, E., Bojanowski, P. and Mikolov, T., 2017. Bag of tricks for efficient text classification. In Proceedings of EACL, pp. 427–431. (Available at: <https://aclanthology.org/E17-2068/>)
- Lemaitre, G., Nogueira, F. and Aridas, C.K., 2017. Imbalanced-learn: A Python toolbox to tackle the curse of imbalanced datasets in machine learning. Journal of Machine Learning Research, 18(17), pp. 1–5. (Available at: <https://jmlr.org/papers/v18/16-365.html>)

- Manning, C.D., Raghavan, P. and Schütze, H., 2008. Introduction to Information Retrieval. Cambridge University Press. (Available at: <https://nlp.stanford.edu/IR-book/>)
- Mozafari, M., Farahbakhsh, R. and Crespi, N., 2020. A BERT-based transfer learning approach for hate speech detection in online social media. In Complex Networks and Their Applications VIII, pp. 928–940. Springer. (Available at: https://link.springer.com/chapter/10.1007/978-3-030-36687-2_77)
- Nobata, C., Tetreault, J., Thomas, A., Mehdad, Y. and Chang, Y., 2016. Abusive language detection in online user content. In Proceedings of the 25th International Conference on World Wide Web, pp. 145–153. (Available at: <https://doi.org/10.1145/2872427.2883062>)
- Park, J.H. and Fung, P., 2017. One-step and two-step classification for abusive language detection on Twitter. In Proceedings of the First Workshop on Abusive Language Online, pp. 41–45. (Available at: <https://aclanthology.org/W17-3006/>)
- Sun, Y., Wong, A.K.C. and Kamel, M.S., 2009. Classification of imbalanced data: A review. International Journal of Pattern Recognition and Artificial Intelligence, 23(4), pp. 687–719. (Available at: <https://doi.org/10.1142/S0218001409007326>)
- Wang, S. and Manning, C.D., 2012. Baselines and bigrams: Simple, good sentiment and topic classification. In Proceedings of ACL, pp. 90–94. (Available at: <https://aclanthology.org/P12-2018/>)
- Waseem, Z. and Hovy, D., 2016. Hateful symbols or hateful people? Predictive features for hate speech detection on Twitter. In Proceedings of the NAACL Student Research Workshop, pp. 88–93. (Available at: <https://aclanthology.org/N16-2013/>)
- Weiss, G.M., 2004. Mining with rarity: A unifying framework. ACM SIGKDD Explorations Newsletter, 6(1), pp. 7–19. (Available at: <https://doi.org/10.1145/1007730.1007734>)
- Zhang, Z., Robinson, D. and Tepper, J., 2018. Detecting hate speech on Twitter using a convolution-GRU based deep neural network. In European Semantic Web Conference, pp. 745–760. Springer. (Available at: https://link.springer.com/chapter/10.1007/978-3-319-93417-4_48)

APPENDIX: ANNOTATED SOURCE CODE

The following subsections present the complete project source code, divided thematically. Each subsection includes explanatory annotations linking the implementation to the methodology.

A. Environment Setup and Imports

Libraries installed and imported at the outset of the notebook. The datasets library provides access to BeaverTails; imbalanced-learn provides RandomUnderSampler; nltk provides stopwords and lemmatisation; tensorflow provides the Keras deep learning framework.

```
import pandas as pd, numpy as np, re, nltk
from datasets import load_dataset
from nltk.corpus import stopwords
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.pipeline import Pipeline
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from imblearn.under_sampling import RandomUnderSampler
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from sklearn.metrics import roc_auc_score
from sklearn.preprocessing import label_binarize
```

B. Dataset Loading and Labelling

The BeaverTails dataset is loaded from Hugging Face. The assign_multi_class_label function implements the four-class priority hierarchy.

```
train_dataset = load_dataset('PKU-Alignment/BeaverTails', split='330k_train')
test_dataset = load_dataset('PKU-Alignment/BeaverTails', split='330k_test')
train_df = train_dataset.to_pandas()
test_df = test_dataset.to_pandas()

def assign_multi_class_label(row):
    cat = row["category"]
    if any([cat.get("violence,aiding_and_abetting,incitement", False),
            cat.get("terrorism,organized_crime", False),
            cat.get("self_harm", False),
            cat.get("child_abuse", False),
            cat.get("sexually_explicit,adult_content", False)]): return 0
    if any([cat.get("non_violent_unethical_behavior", False),
            cat.get("misinformation_regarding_ethics,laws_and_safety", False),
```

```

        cat.get("privacy_violation", False),
        cat.get("financial_crime,property_crime,theft", False),
        cat.get("drug_abuse,weapons,banned_substance", False)]: return 1
    if any([cat.get("discrimination,stereotype,injustice", False),
            cat.get("hate_speech,offensive_language", False),
            cat.get("controversial_topics,politics", False),
            cat.get("animal_abuse", False)]): return 2
    return 3

train_df["label"] = train_df.apply(assign_multi_class_label, axis=1)
test_df["label"] = test_df.apply(assign_multi_class_label, axis=1)

```

C. Feature Construction and Text Cleaning

The `full_text` column concatenates prompt and response. The `clean_text` function applies lowercasing, URL removal, punctuation removal, stopword filtering (preserving not/no/how), and lemmatisation.

```

train_df['full_text'] = train_df['prompt'] + ' ' + train_df['response']
test_df['full_text'] = test_df['prompt'] + ' ' + test_df['response']

from nltk.stem import WordNetLemmatizer
lemmatizer = WordNetLemmatizer()
stop_words = set(stopwords.words('english'))

def clean_text(text):
    text = text.lower()
    text = re.sub(r'http\S+', '', text)
    text = re.sub(r'[^a-zA-Z\s]', '', text)
    tokens = text.split()
    tokens = [lemmatizer.lemmatize(w) for w in tokens
              if w not in stop_words or w in ['not', 'no', 'how']]
    return ' '.join(tokens)

train_df['clean_prompt'] = train_df['full_text'].apply(clean_text)
test_df['clean_prompt'] = test_df['full_text'].apply(clean_text)

```

D. Class Balancing and Data Splitting

`RandomUnderSampler` produces a class-balanced training subset. The balanced data is split 80/20 using stratified sampling. A balanced test set is also created for per-class evaluation.

```

X_train = train_df['clean_prompt']; y_train = train_df['label']
X_test = test_df['clean_prompt']; y_test = test_df['label']

rus = RandomUnderSampler(random_state=42)
X_resampled, y_resampled = rus.fit_resample(X_train.to_frame(), y_train)
X_resampled = X_resampled['clean_prompt']

```

```
X_train_final, X_val, y_train_final, y_val = train_test_split(
    X_resampled, y_resampled, test_size=0.2,
    random_state=42, stratify=y_resampled)

# Balanced test set for per-class analysis
X_test_bal, y_test_bal = rus.fit_resample(X_test.to_frame(), y_test)
X_test_bal = X_test_bal['clean_prompt']
```

E. Logistic Regression Model

The LR pipeline uses (1,3) n-gram TF-IDF vectorisation. Training is on the balanced subset. Predictions and class probabilities are generated on the imbalanced test set.

```
logistic_model = Pipeline([
    ('tfidf', TfidfVectorizer(ngram_range=(1,3))),
    ('model', LogisticRegression(max_iter=10000, verbose=1))
])
logistic_model.fit(X_train_final, y_train_final)

log_pred = logistic_model.predict(X_test)
log_probs = logistic_model.predict_proba(X_test)
print('Logistic Regression')
print(classification_report(y_test, log_pred))
```

F. Linear SVC Model

LinearSVC uses a max_features-constrained unigram TF-IDF and class_weight="balanced". Decision function scores are used as proxy probability scores for ROC-AUC.

```
svc_model = Pipeline([
    ('tfidf', TfidfVectorizer(max_features=20000)),
    ('model', LinearSVC(class_weight='balanced'))
])
svc_model.fit(X_train_final, y_train_final)

svc_pred = svc_model.predict(X_test)
svc_scores = svc_model.decision_function(X_test)
print('LinearSVC')
print(classification_report(y_test, svc_pred))
```

G. LSTM Tokenisation and Label Encoding

CRITICAL: The Tokenizer and LabelEncoder are fitted ONLY on X_train_final / y_train_final (the balanced subset). Earlier versions incorrectly fitted on the full X_train, bypassing undersampling.

```
max_words = 20000; max_length = 100
```

```

# Fitted on BALANCED subset only (critical fix)
tokenizer = Tokenizer(num_words=max_words)
tokenizer.fit_on_texts(X_train_final)

X_train_seq = tokenizer.texts_to_sequences(X_train_final)
X_test_seq = tokenizer.texts_to_sequences(X_test)
X_train_pad = pad_sequences(X_train_seq, maxlen=max_length)
X_test_pad = pad_sequences(X_test_seq, maxlen=max_length)

X_test_bal_seq = tokenizer.texts_to_sequences(X_test_bal)
X_test_bal_pad = pad_sequences(X_test_bal_seq, maxlen=max_length)

# Fitted on BALANCED subset only
label_encoder = LabelEncoder()
y_train_lstm = label_encoder.fit_transform(y_train_final)
y_test_lstm = label_encoder.transform(y_test)
y_test_bal_lstm = label_encoder.transform(y_test_bal)

```

H. LSTM Architecture and Training

Architecture: Embedding → LSTM(128) → Dropout(0.5) → Dense(64, relu) → Dense(4, softmax).

EarlyStopping prevents overfitting and restores best weights.

```

from tensorflow.keras.callbacks import EarlyStopping

lstm_model_imb = Sequential([
    Embedding(input_dim=max_words, output_dim=128, input_length=max_length),
    LSTM(128),
    Dropout(0.5),
    Dense(64, activation='relu'),
    Dense(4, activation='softmax')
])
lstm_model_imb.compile(loss='sparse_categorical_crossentropy',
                      optimizer='adam', metrics=['accuracy'])

early_stop = EarlyStopping(monitor='val_loss', patience=5,
                          restore_best_weights=True)
history = lstm_model_imb.fit(
    X_train_pad, y_train_lstm,
    epochs=50, batch_size=8192,
    validation_split=0.2,
    callbacks=[early_stop])

lstm_probs = lstm_model_imb.predict(X_test_pad)
lstm_pred = np.argmax(lstm_probs, axis=1)

```

I. Model Evaluation

Confusion matrices are plotted for all three models on imbalanced and balanced test sets. Full classification reports are printed.

```
from sklearn.metrics import ConfusionMatrixDisplay

y_pred_lr_bal = logistic_model.predict(X_test_bal)
y_pred_svc_bal = svc_model.predict(X_test_bal)
y_pred_lstm_bal = np.argmax(lstm_model_imb.predict(X_test_bal_pad), axis=1)

def plot_cm(y_true, y_pred, title):
    ConfusionMatrixDisplay.from_predictions(y_true, y_pred)
    plt.title(title); plt.show()

plot_cm(y_test, log_pred, 'LR - Imbalanced')
plot_cm(y_test_bal, y_pred_lr_bal, 'LR - Balanced')
plot_cm(y_test, svc_pred, 'SVC - Imbalanced')
plot_cm(y_test_bal, y_pred_svc_bal, 'SVC - Balanced')
plot_cm(y_test_lstm, lstm_pred, 'LSTM - Imbalanced')
plot_cm(y_test_bal_lstm, y_pred_lstm_bal, 'LSTM - Balanced')
```

J. ROC-AUC and Inference Function

ROC-AUC computed with one-vs-rest binarisation. The `predict_new_text` function provides real-time inference across all three models.

```
y_test_bin = label_binarize(y_test, classes=[0,1,2,3])
log_auc = roc_auc_score(y_test_bin, log_probs, multi_class='ovr')
svc_auc = roc_auc_score(y_test_bin, svc_scores, multi_class='ovr')
lstm_auc = roc_auc_score(y_test_bin, lstm_probs, multi_class='ovr')
print('LR:', log_auc, ' SVC:', svc_auc, ' LSTM:', lstm_auc)

label_names = ['Severe Harm', 'Non-violent Harm', 'Social/Contextual Harm', 'Safe']

def predict_new_text(text):
    clean_txt = clean_text(text)
    log_p = logistic_model.predict_proba([clean_txt])[0]
    svc_idx = svc_model.predict([clean_txt])[0]
    seq = tokenizer.texts_to_sequences([clean_txt])
    pad = pad_sequences(seq, maxlen=max_length)
    lstm_p = lstm_model_imb.predict(pad, verbose=0)[0]
    return {
        'LR': {'label': label_names[np.argmax(log_p)], 'probs': log_p},
        'SVC': {'label': label_names[svc_idx]},
        'LSTM': {'label': label_names[np.argmax(lstm_p)], 'probs': lstm_p}
    }
```